ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Inductive Definitions

We have introduced the λ-calculus with a degree of informality in the way we have described the syntax using BNF, β-reduction by example, and reasoned about reduction strategies through intuition.

As we move forward, we need a more solid framework to enable us to:

1. Define relations (evaluation, typing) and the corresponding evaluation and typing systems, formally and unambiguously
2. Reason about such relations formally, and implement the systems effectively.

Inductive definitions provide exactly this framework. They give us a constructive way to specify sets and relations, together with a powerful reasoning principle — *rule induction* — that we will use extensively when we study such relations.

Judgments and Judgment Forms

A *judgment form* is a property of terms, or a relation among terms. We write J for a judgment form. A *judgment* is an assertion that specific terms have that property or stand in that relation; we write $J(a)$ for the judgment that J holds of the term a, and $J(a, b)$ when J relates two terms.

Examples of judgment forms and their instances:

Judgment form	Instance	Reading
$- \in \mathsf{nat}$	$\mathsf{succ}(0) \in \mathsf{nat}$	$\mathsf{succ}(0)$ is a natural number
$- : -$	$M : T$	M has type T
$- \to -$	$M \to N$	M reduces to N

We often write judgments in their natural notation ($n\ \mathsf{nat}$, $M \to N$) rather than the explicit $J(a)$ form. The $J(a)$ notation becomes useful when stating general principles.

Inductive Rules

An inductive definition consists of a family of *rules* for deriving judgments of a given form. Rules have the general structure:

$$\frac{J(a_1) \quad \cdots \quad J(a_k)}{J(a)}$$

The judgments above the line are *premises*; the judgment below is the *conclusion*. A rule with no premises ($k = 0$) is an *axiom*; otherwise it is an *inference rule*.

A rule states that the premises are *sufficient conditions* for the conclusion:

- To establish $J(a)$, it is enough to establish $J(a_1), \ldots, J(a_k)$.
- Equivalently: if $J(a_1), \ldots, J(a_k)$ hold, then $J(a)$ holds.

An axiom states that its conclusion holds unconditionally.

Derivations

A *derivation* is a finite composition of rules, starting from axioms and ending with a conclusion. We can visualize a derivation as a tree (read bottom-up), where each node is a rule and its children are derivations of the rule's premises.

$$\frac{\dfrac{J(a_{1,1}) \quad \cdots \quad J(a_{1,n_1})}{J(a_1)} \quad \cdots \quad \dfrac{J(a_{k,1}) \quad \cdots \quad J(a_{k,n_k})}{J(a_k)}}{J(a)}$$

To show that a judgment $J(a)$ holds, we construct a derivation with $J(a)$ at the root.

Example 1: Natural Numbers

The natural numbers can be defined inductively by two rules for the judgment form $- \in \mathsf{nat}$:

$$\frac{}{0 \in \mathsf{nat}} \qquad\qquad \frac{n \in \mathsf{nat}}{\mathsf{succ}(n) \in \mathsf{nat}}$$

The first rule is an axiom: zero is a natural number unconditionally. The second is a *rule scheme* — it stands for infinitely many rules, one for each n. Read together, these rules define Nat as the smallest set containing 0 and closed under successor.

The numeral 2 is the term $\mathsf{succ}(\mathsf{succ}(0))$. Here is its derivation:

$$\dfrac{\dfrac{\dfrac{}{0 \in \mathsf{nat}}}{\mathsf{succ}(0) \in \mathsf{nat}}}{\mathsf{succ}(\mathsf{succ}(0)) \in \mathsf{nat}}$$

This derivation is *evidence* that 2 nat. Crucially, there is no derivation for -1 nat or $\frac{1}{2}$ nat — these are not natural numbers precisely because no derivation exists.

Example 2: Lambda Terms

BNF grammars are inductive definitions in disguise. The grammar

$$M, N ::= x \mid \lambda x.M \mid M\ N$$

is equivalent to the following inductive definition for the judgment form $- \in \mathsf{Lam}$:

$$\dfrac{}{x \in \mathsf{Lam}} \qquad \dfrac{M \in \mathsf{Lam}}{\lambda x.M \in \mathsf{Lam}} \qquad \dfrac{M \in \mathsf{Lam} \quad N \in \mathsf{Lam}}{M\ N \in \mathsf{Lam}}$$

The first rule (implicitly, for each variable x) says variables are terms. The second says that if M is a term, so is $\lambda x.M$. The third says that if M and N are both terms, so is their application $M\ N$.

The Meaning of an Inductive Definition

An inductive definition specifies the **smallest** relation that is **closed under the rules**. Spelling out these two qualifications:

- **Closed under the rules** is a **sufficient condition**: to show that a judgment holds, it is enough to derive it by composing the rules.
- **Smallest** is a **necessary condition**: if a judgment $J(a)$ holds, then it *must* have been derived by composing the rules.

Said differently: a judgment holds *if and only if* it has a derivation.

The Principle of Rule Induction

Based on this interpretation, we derive the principle of rule induction to reason about inductively defined relations.

To illustrate, we start with the principle of ordinary **mathematical induction** on natural numbers:

> To prove that a property P holds for all natural numbers, it suffices to show
>
> 1. $P(0)$;
>
> 2. For any n, if $P(n)$ then $P(n+1)$.

Rule induction generalizes this to any inductively defined judgment form.

The Principle of Rule Induction. To show that a property P holds for all elements of the set inductively defined by a judgment form J it suffices to show:

1. **Base cases**: for all axioms $J(a)$, $P(a)$ holds.
2. **Inductive Cases**: for all inference rules $\dfrac{J(a_1) \quad \cdots \quad J(a_k)}{J(a)}$, if $P(a_1), \ldots, P(a_k)$ all hold (the *inductive hypotheses*), then $P(a)$ holds (the *inductive conclusion*).

When these conditions hold, we say that P *respects* all the rules (both axioms and inference rules) defining J
J.

Rule Induction on Natural Numbers

Applying rule induction to our definition of $-\ \mathsf{nat}$, we must show:

1. $P(0)$ holds (showing that P respects the axiom for 0)
2. For any n, if $P(n)$ holds, then $P(\mathsf{succ}(n))$ holds (respecting the successor rule)

This is precisely ordinary mathematical induction — rule induction subsumes it as a special case.

Rule Induction on Lambda Terms

To prove that a property P holds of all λ-terms, we show:

1. $P(x)$ holds for every variable x (respecting the axiom for variables)
2. If $P(M)$ holds, then $P(\lambda x.M)$ holds (respecting the inference rule for lambda abstractions)
3. If $P(M)$ and $P(N)$ both hold, then $P(M\ N)$ holds (respecting the inference rule for applications)

This is sometimes called *structural induction* on terms.

A worked example: Proof by rule induction

Claim. For all $n \in \mathsf{nat}$, we have $n + 0 = n$, where addition is defined by:

1. $0 + m = m$
2. $\mathsf{succ}(n) + m = \mathsf{succ}(n + m)$

Proof. Let $P(n)$ be the property $n + 0 = n$. We proceed by rule induction on showing that P respects the rules defining $- \in \mathsf{nat}$

- For the axiom $0 \in \mathsf{nat}$, we must show $P(0)$, i.e. $0 + 0 = 0$. This holds by 1. . ✓
- For the successor rule, assuming the induction hypothesis $P(n)$ i.e., $n + 0 = n$, we must show $P(\mathsf{succ}(n))$, i.e.
  $\mathsf{succ}(n) + 0 = \mathsf{succ}(n)$.
  We have
  $$
  \hspace*{-10cm}
  \begin{array}{llll}
  \mathsf{succ}(n) + 0 & = &\mathsf{succ}(n + 0) & \text{by\ 2.} \\
  & = &\mathsf{succ}(n)& \text{by the induction hypothesis}
  \end{array}
  $$

Rule Induction and Reduction in the λ-Calculus

In our presentation of the λ-calculus, we described β-reduction informally: a redex $(\lambda x.M)\, N$ reduces to $[N/x]M$, and we may reduce any redex occurring anywhere in a term. e also discussed various *strategies*, that determine which redex to reduce when multiple choices exist.
Inductive definitions let us make these strategies precise. We start with full β-reduction

Full β-Reduction

Full β-reduction lets us choose the redex nondeterministically. This is captured by the following rules for the judgment form $- \to -$:

$$\frac{}{(\lambda x.M)\, N \to [N/x]M}\,(\beta)$$

$$\frac{M \to M'}{M\, N \to M'\, N}\,(\text{app-L}) \qquad \frac{N \to N'}{M\, N \to M\, N'}\,(\text{app-R}) \qquad \frac{M \to M'}{\lambda x.M \to \lambda x.M'}\,(\text{abs})$$

The axiom (β) captures the computational content of β-reduction. The three inference rules allow reduction to occur *inside* terms: in the function position of an application (app-L), in the argument position (app-R), or under a λ-abstraction (abs).

The rules formalize the notion that β-reduction may occur anywhere in a term. For example, to show that $(\lambda x.x)\,((\lambda y.y)\,z) \to (\lambda x.x)\,z$, we build the derivation:

$$\dfrac{\dfrac{}{(\lambda y.y)\,z \to z}(\beta)}{(\lambda x.x)\,((\lambda y.y)\,z) \to (\lambda x.x)\,z}(\text{app-R})$$

Rule Induction on Full β-Reduction To prove that a property P holds of all β-reduction steps $M \to M'$, we show:

1. $P((\lambda x.M)\,N, [N/x]M)$ holds for any M and N (respecting rule β)
2. If $P(M, M')$ holds, then $P(M\,N, M'\,N)$ holds (respecting app-L)
3. If $P(N, N')$ holds, then $P(M\,N, M\,N')$ holds (respecting app-R)
4. If $P(M, M')$ holds, then $P(\lambda x.M, \lambda x.M')$ holds (respecting abs)

Evaluation Strategies

We now look at the strategies — call-by-name, call-by-value, normal order, applicative order — that we have so far introduced relying on intuitive definitions of "leftmost-outermost", "reduce arguments first", and so on.

With inductive definition, each strategy is represented by a judgment form $M \to M'$ defined by a specific set of rules. The rules themselves encode the strategy: which redexes may fire, and under what conditions. Different strategies correspond to different rule sets for the same judgment form.

The key technique is to use *auxiliary judgments* that characterize the structure of terms, allowing us to control evaluation order precisely.

Values. A *value* is a λ-abstraction. The judgment form $-\ \mathsf{val}$ is defined by:

$$\dfrac{}{\lambda x.M\ \mathsf{val}}$$

Not an Abstraction. The judgment form $-\ \mathsf{notabs}$ holds when M is not a λ-abstraction:

$$\dfrac{}{x\ \mathsf{notabs}} \qquad \dfrac{}{M\,N\ \mathsf{notabs}}$$

Normal Form. A term is in *normal form* if no β-redex occurs anywhere within it. The judgment form $-\ \mathsf{nf}$ is defined by:

$$\frac{M\ \mathsf{nf}}{\lambda x.M\ \mathsf{nf}} \qquad \frac{}{x\ \mathsf{nf}} \qquad \frac{M\ \mathsf{nf} \quad M\ \mathsf{notabs} \quad N\ \mathsf{nf}}{M\ N\ \mathsf{nf}}$$

The third rule is the key: an application $M\ N$ is in normal form when both M and N are in normal form *and* M is not a λ-abstraction (otherwise $M\ N$ would be a redex).

Normal Order

Normal order reduction contracts the *leftmost-outermost* redex and reduces under λ-abstractions. It proceeds by first reducing to weak head normal form, then recursively normalizing subterms.

$$\frac{}{(\lambda x.M)\ N \to_{\mathsf{no}} [N/x]M}(\beta)$$

$$\frac{M\ \mathsf{notabs} \quad M \to_{\mathsf{no}} M'}{M\ N \to_{\mathsf{no}} M'\ N}(\text{ap-L})$$

$$\frac{M\ \mathsf{nf} \quad M\ \mathsf{notabs} \quad N \to_{\mathsf{no}} N'}{M\ N \to_{\mathsf{no}} M\ N'}(\text{ap-R})$$

$$\frac{M \to_{\mathsf{no}} M'}{\lambda x.M \to_{\mathsf{no}} \lambda x.M'}(\text{abs})$$

Reading the rules:

- **(β)**: A redex at the root reduces immediately — this captures "outermost first".
- **(ap-L)**: Reduce in the function position when it is not a λ-abstraction (if it were, rule β would apply instead).
- **(ap-R)**: Reduce in the argument only when the function is in normal form and not a λ-abstraction (so no redex can ever form).
- **(abs)**: Reduce under λ-abstractions.

Applicative Order

Applicative order reduction reduces the *leftmost-innermost* redex and under λ-abstractions. It normalizes subterms before performing β-reduction.

$$\frac{M\ \mathsf{nf} \quad N\ \mathsf{nf}}{(\lambda x.M)\ N \to_{\mathsf{ao}} [N/x]M}(\beta)$$

$$\frac{M\ \mathsf{notabs} \quad M \to_{\mathsf{ao}} M'}{M\ N \to_{\mathsf{ao}} M'\ N}(\text{ap-L})$$

$$\frac{M\ \mathsf{nf} \quad M\ \mathsf{notabs} \quad N \to_{\mathsf{ao}} N'}{M\ N \to_{\mathsf{ao}} M\ N'}(\text{ap-R})$$

$$\frac{M \to_{\mathsf{ao}} M'}{(\lambda x.M)\, N \to_{\mathsf{ao}} (\lambda x.M')\, N}\,(\text{ap-L-abs})$$

$$\frac{M \;\mathsf{nf} \quad N \to_{\mathsf{ao}} N'}{(\lambda x.M)\, N \to_{\mathsf{ao}} (\lambda x.M)\, N'}\,(\text{ap-R-abs})$$

$$\frac{M \to_{\mathsf{ao}} M'}{\lambda x.M \to_{\mathsf{ao}} \lambda x.M'}\,(\text{abs})$$

Reading the rules:

- **(β)**: A redex fires only when *both* the body and argument are in normal form — this captures "innermost first".
- **(ap-L), (ap-R)**: When the function is not a λ-abstraction, normalize left-to-right.
- **(ap-L-abs), (ap-R-abs)**: When the function *is* a λ-abstraction, we must normalize inside the λ and in the argument before firing the redex.
- **(abs)**: Reduce under λ-abstractions (when not in an application).

Call-by-Name

Call-by-name is the strategy that underlies evaluation in languages like Haskell. It corresponds to normal order, but stops at weak head normal form — it does not reduce under λ-abstractions or inside arguments. This restriction makes it suitable for implementation: a language runtime never inspects the body of a function until it is applied, and never evaluates an argument until it is actually used.

$$\frac{}{(\lambda x.M)\, N \to_{\mathsf{n}} [N/x]M}\,(\beta)$$

$$\frac{M \to_{\mathsf{n}} M'}{M\, N \to_{\mathsf{n}} M'\, N}\,(\text{ap})$$

Call-by-Value

Much in the same way that call-by-name corresponds to normal order, call-by-value corresponds to applicative order — both call-by-value and applicative order evaluate arguments before substitution, but call-by-value stops at weak head normal form rather than reducing under λ-abstractions. This is the strategy underlying strict languages like ML, Scala, and most mainstream programming languages.

$$\frac{N \;\mathsf{val}}{(\lambda x.M)\, N \to_{\mathsf{v}} [N/x]M}\,(\beta)$$

$$\frac{M \to_{\mathsf{v}} M'}{M\, N \to_{\mathsf{v}} M'\, N}\,(\text{ap-L})$$

$$\frac{M\ \mathsf{val} \quad N \to_{\mathsf{v}} N'}{M\,N \to_{\mathsf{v}} M\,N'}(\text{ap-R})$$

Summary of Strategies

Strategy	Redex Selection	Under λ?	Normalizing?	Closed terms?
Normal order	Leftmost-outermost	Yes	Yes	No
Applicative order	Leftmost-innermost	Yes	No	No
Call-by-name	Leftmost-outermost	No	Yes (to whnf)	Yes
Call-by-value	Leftmost-innermost	No	No	Yes

Remarks

- *normalizing* means: if a normal form exists, this strategy will find it.
- *closed terms*: call-by-name and call-by-value assume closed terms. A term like $x\,M$ with a free variable in function position, is stuck — there is no applicable rule because x is neither a λ-abstraction nor can it step. This reflects the reality of programming languages: a well-formed program is closed, and attempting to apply an undefined identifier is an error.

Key Properties

Theorem (Determinacy). Each evaluation strategy defines a deterministic transition: if $M \to M'$ and $M \to M''$, then $M' =_\alpha M''$, where "=_\alpha" is equality up to α equivalence.

Proof idea. By rule induction, showing the rules are mutually exclusive. The auxiliary judgments (val, nf, notabs) partition the space of terms so that exactly one rule applies. □

Theorem (Normal Order Normalization). If M has a normal form, then repeated application of $\to_{\mathsf{no}}$ reaches it.

This is a consequence of the *standardization theorem* for the λ-calculus.

Counterexample (Applicative Order Divergence). Let $\Omega = (\lambda x.x\,x)(\lambda x.x\,x)$. The term $(\lambda x.y)\,\Omega$ has normal form y, but applicative order diverges:

$$(\lambda x.y)\,\Omega \to_{\mathsf{ao}} (\lambda x.y)\,\Omega \to_{\mathsf{ao}} \cdots$$

because it tries to normalize Ω before firing the redex.

Exercises

Exercise 1: Expressions

Recall the expression language from the first lecture:

$$E ::= n \mid E + E \mid E \times E$$

Express this syntax as an inductive definition for the judgment form $- \in \mathsf{Exp}$.

Exercise 2: A Custom Set

Let the set Q be defined inductively as follows:

$$\frac{}{8 \in \mathsf{Q}}(\mathrm{R1}) \qquad \frac{}{5 \in \mathsf{Q}}(\mathrm{R2}) \qquad \frac{a \in \mathsf{Q} \quad b \in \mathsf{Q}}{a + b + 1 \in \mathsf{Q}}(\mathrm{R3})$$

(a) Which rules are axioms and which are inference rules?

(b) Give a derivation showing that $14 \in \mathsf{Q}$.

(c) Give a derivation showing that $20 \in \mathsf{Q}$.

(d) State the principle of rule induction for Q. That is, if you want to prove that a property $P(a)$ holds for all $a \in \mathsf{Q}$, what must you show?

(e) Is $2 \in \mathsf{Q}$? Either give a derivation, or prove that no derivation exists. (*Hint:* Find a property that holds of all elements of Q but not of 2.)

Exercise 3: Properties of Lambda Terms

Using rule induction on lambda terms, prove that every λ-term has at least one variable occurrence (either free or bound).

Exercise 4: Reduction Preserves Size

Define the *size* of a λ-term inductively:

- $size(x) = 1$
- $size(\lambda x.M) = 1 + size(M)$

- $size(M\ N) = 1 + size(M) + size(N)$

Does β-reduction always decrease size? Always increase size? Neither? Justify your answer with examples or a proof.

Exercise 5: Normal Forms

Prove by rule induction that if $M\ \mathsf{nf}$, then there is no M' such that $M \to_{\mathsf{no}} M'$.

Credits

The treatment of inductive definitions and rule induction follows Robert Harper, *Practical Foundations for Programming Languages* (2nd ed.), Cambridge University Press, 2016.